

StackFlow - الكتاب الكامل

دليل شامل لفهم مشروع TikTok Manager من الألف إلى الياء

كتاب StackFlow - دليل المشروع الكامل

النسخة: 1.0

التاريخ: 13 يوليو 2026

الوصف: شرح تفصيلي كامل لمشروع TikTok Manager (StackFlow) من الألف إلى الياء

🕒 فهرس المحتويات

الجزء الأول: أساسيات تطوير الويب

1. ما هو تطوير الويب؟
2. الفرق بين Frontend و Backend و Database
3. كيف تتواصل الأجزاء مع بعضها؟
4. ما هو API و REST؟
5. بيئة التطوير والأدوات

الجزء الثاني: نظرة عامة على المشروع

6. ما هو StackFlow؟
7. المكدس التكنولوجي الكامل
8. هيكل المشروع الكامل
9. كيف يعمل النظام ككل؟

الجزء الثالث: قاعدة البيانات (PostgreSQL)

10. ما هي قاعدة البيانات؟
11. لماذا PostgreSQL؟
12. جدول users
13. جدول posts
14. جدول daily_stats
15. جدول personal_access_tokens
16. العلاقات بين الجداول
17. فهم الـ Migrations

الجزء الرابع: الـ باك إند (Laravel)

18. ما هو Laravel؟
19. نمط MVC
20. ملف routes/api.php
21. AuthController
22. DashboardController
23. PostController
24. TikTokAuthController
25. الموديلات (Models)
26. Sanctum للمصادقة
27. TikTokService
28. رفع الملفات إلى Cloudinary
29. CORS

الجزء الخامس: الفرونت إند (React)

30. ما هو React؟
31. main.tsx
32. App.tsx
33. types/index.ts

services/ (api.ts, auth.ts, dashboard.ts, posts.ts) .34

store/index.ts (Zustand) .35

hooks/useTranslation.ts .36

components/ (Sidebar, StatsCard) .37

Tailwind CSS .38

الجزء السادس: شرح الصفحات بالتفصيل

Login صفحة .39

Dashboard صفحة .40

Posts صفحة .41

CreatePost صفحة .42

Settings صفحة .43

44. تدفق البيانات الكامل

الجزء السابع: النشر والتشغيل

45. تحضير الكود للنشر

46. نشر الفرونت إند على Vercel

47. نشر الباك إند على Render

48. Render على PostgreSQL

49. Cloudinary

50. TikTok Sandbox

الجزء الأول: أساسيات تطوير الويب

1. ما هو تطوير الويب؟

تطوير الويب (Web Development) هو عملية بناء وتطوير مواقع الإنترنت. أي موقع ويب يتكون من ثلاثة أجزاء رئيسية:



تشبيه بسيط

تخيل أنك تذهب إلى مطعم:

الجزء	المطعم	StackFlow
الواجهة (Frontend)	قائمة الطعام + شكل المطعم	React (يعرض الصفحات)
الخادم (Backend)	الطباخ الذي يحضر الطعام	Laravel (يعالج الطلبات)
قاعدة البيانات	الثلاجة حيث تُحفظ المكونات	PostgreSQL (تخزن البيانات)

الزبون (المستخدم) يرى فقط قائمة الطعام (الواجهة)، يطلب طبقًا، الطباخ (الخادم) يجهز الطلب باستخدام المكونات من الثلاجة (قاعدة البيانات)، ثم يقدم الطبق للزبون.

أنواع تطبيقات الويب

1. مواقع ثابتة (Static): مثل صفحات التعريف، لا تتغير حسب المستخدم
 2. تطبيقات ديناميكية (Dynamic): مثل فيسبوك، تتغير حسب المستخدم والبيانات
 3. تطبيقات صفحة واحدة (SPA): مثل StackFlow، كل الصفحات في ملف واحد
- مشروع StackFlow هو تطبيق صفحة واحدة (SPA) مع باك إند منفصل. هذا يعني أن الواجهة والخادم يعملان بشكل مستقل ويتواصلان عبر API.

2. الفرق بين Frontend و Backend و Database

الفرونت إند (Frontend)

الفرونت إند هو كل ما يراه المستخدم ويتفاعل معه في المتصفح.

التقنيات المستخدمة في StackFlow:

شرح مبسط	الدور	التقنية
يقسم الصفحة إلى مكونات صغيرة	بناء واجهة المستخدم	React
يمنع الأخطاء قبل التشغيل	إضافة أنواع للكود	TypeScript
يحول الكود إلى ملفات للمتصفح	أداة بناء وتشغيل	Vite
مباشرة HTML داخل CSS أكواد	التنسيق	Tailwind CSS
مخزن مركزي للبيانات	حفظ البيانات المشتركة	Zustand
HTTP يرسل ويستقبل طلبات	التواصل مع الخادم	Axios
رسوم بيانية جميلة وسهلة	رسم المخططات البيانية	Recharts
يغير الصفحة بدون إعادة تحميل	التنقل بين الصفحات	React Router

الباك إند (Backend)

الباك إند هو الخادم الذي يعمل في الخلفية. المستخدم لا يراه مباشرة.

التقنيات المستخدمة:

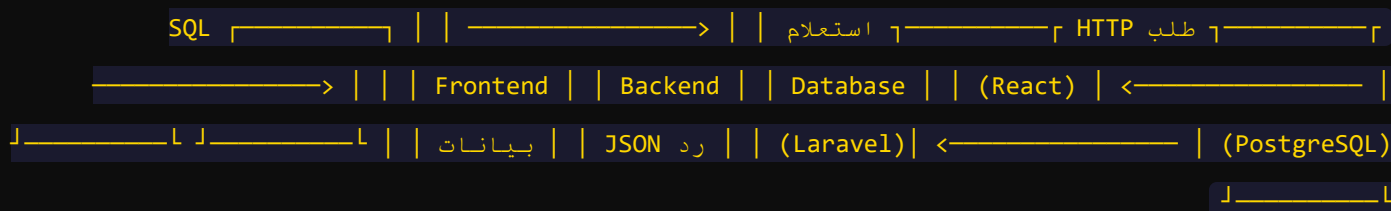
الدور	التقنية
PHP (Framework) إطار عمل	Laravel 13
لغة البرمجة	PHP 8.3+
(API tokens) نظام المصادقة	Sanctum
التعامل مع قاعدة البيانات	Eloquent ORM
للخدمات الخارجية HTTP طلبات	Guzzle
تخزين الملفات (صور/فيديو)	Cloudinary

قاعدة البيانات (Database)

قاعدة البيانات هي المكان الذي تُحفظ فيه كل البيانات بشكل منظم.

StackFlow يستخدم **PostgreSQL**، وهي قاعدة بيانات علائقية (Relational Database). البيانات تخزن في جداول (**Tables**) مثل جدول إكسل.

3. كيف تتواصل الأجزاء مع بعضها؟



الخطوات بالتفصيل:

1. المستخدم يضغط على "Login" في الفرونت إند
2. الفرونت إند يرسل طلب HTTP POST إلى الباك إند
3. الباك إند يستقبل الطلب، يتحقق من البيانات
4. الباك إند يتصل بقاعدة البيانات: `SELECT * FROM users WHERE email = ?`
5. قاعدة البيانات ترجع بيانات المستخدم
6. الباك إند يتحقق من كلمة المرور
7. الباك إند يرجع رد JSON للفرونت إند
8. الفرونت إند يستقبل الرد ويظهر اسم المستخدم

4. ما هو API و REST؟

API (واجهة برمجة التطبيقات)

API = Application Programming Interface. هو وسيلة التواصل بين التطبيقات المختلفة.

في مشروعنا:

- الفرونت إند (React) يتواصل مع الباك إند (Laravel) عبر API
- الباك إند يتواصل مع TikTok عبر API

REST = Representational State Transfer. هو نمط معين لبناء APIs.

طرق (Methods) HTTP

StackFlow مثال في	المعنى	الطريقة
جلب قائمة المنشورات	قراءة بيانات	GET
تسجيل دخول، إنشاء منشور	إنشاء جديد	POST
حذف منشور	حذف	DELETE

مسارات API في StackFlow

POST	/api/login	تسجيل الدخول <-
POST	/api/logout	تسجيل الخروج <-
GET	/api/user	معلومات المستخدم <-
GET	/api/auth/tiktok/redirect	TikTok رابط توثيق <-
GET	/api/auth/tiktok/callback	TikTok استقبال رد <-
GET	/api/dashboard/stats	إحصائيات <-
GET	/api/dashboard/daily-stats	إحصائيات يومية <-
GET	/api/posts	قائمة المنشورات <-
POST	/api/posts	إنشاء منشور <-
DELETE	/api/posts/{id}	حذف منشور <-

5. بيئة التطوير والأدوات

الأوامر الأساسية

```
# تشغيل الفرونت إند
cd TikTokManager

npm install          # تحميل المكتبات
npm run dev          # http://localhost:5173

# تشغيل الباك إند
cd backend

composer install     # تحميل مكتبات PHP
cp .env.example .env # نسخ ملف الإعدادات
php artisan key:generate # توليد مفتاح التشفير
php artisan serve     # http://localhost:8000
```

الجزء الثاني: نظرة عامة على المشروع

6. ما هو StackFlow؟

StackFlow هو تطبيق ويب لإدارة حسابات **TikTok**. يسمح لك بـ:

- ربط حساب TikTok الخاص بك عبر OAuth
- مشاهدة إحصائيات حسابك (المشاهدات، الإعجابات، المتابعين)
- إدارة المنشورات (عرض، إنشاء، حذف)
- رؤية رسم بياني للإحصائيات اليومية
- تغيير الإعدادات الشخصية واللغة

لمن هذا التطبيق؟

- منشئي المحتوى على TikTok
- المسوقين الذين يديرون حسابات متعددة
- الوكالات التي تدير حسابات عملاء

7. المكّس التكنولوجي الكامل

الفرونت إند

React 18 + TypeScript + Vite 5 + Tailwind CSS 4 + React Router 6 + Zustand 5 + Axios + Zod +
Recharts

الباك إند

Laravel 13 + PHP 8.3 + Sanctum + Eloquent ORM + Guzzle + Cloudinary SDK

قاعدة البيانات

PostgreSQL 16

أدوات التشغيل

Vercel (Frontend) + Render (Backend + DB) + Cloudinary (Files) + GitHub (Code)

8. هيكل المشروع الكامل

```

TikTokManager/
├─ index.html           # الرئيسية HTML صفحة
├─ package.json         # إعدادات npm
├─ vite.config.ts       # إعدادات Vite
├─ .gitignore           # الملفات المتجاهلة
├─ ai_changes           # سجل تغييرات AI
├─
├─ src/                 # الفرونت إند
│   ├─ main.tsx         # نقطة الدخول
│   ├─ App.tsx          # المكون الرئيسي (التوجيه)
│   ├─ index.css        # Tailwind + تنسيقات
│   │
│   ├─ types/index.ts   # User, Post, Stats, DailyStats
│   ├─ services/        # api.ts, auth.ts, dashboard.ts, posts.ts
│   ├─ store/index.ts   # Zustand stores
│   ├─ hooks/useTranslation.ts # نظام الترجمة
│   ├─ config/          # colors.ts + languages/
│   ├─ components/      # Sidebar.tsx, StatsCard.tsx
│   └─ pages/           # Login, Dashboard, Posts, CreatePost, Settings
├─
├─ backend/             # الباك إند
│   ├─ routes/api.php   # API مسارات
│   ├─ app/Http/Controllers/Api/ # AuthController, DashboardController, PostController, TikTokA
│   ├─ app/Models/      # User, Post, DailyStat
│   ├─ app/Services/    # TikTokService
│   ├─ config/          # cors.php, sanctum.php, cloudinary.php
│   └─ database/migrations/ # ملفات إنشاء الجداول
├─
├─ dist/                # نسخة الإنتاج من الفرونت إند
└─ scripts/md-to-pdf.js # Markdown تحويل إلى PDF

```

9. كيف يعمل النظام ككل؟

مثال: مستخدم يريد رؤية المنشورات

1. المتصفح -> <https://stackflow.vercel.app/posts>
2. React Router -> يجد المسار /posts
3. <Posts /> يعرض مكون
4. Posts.tsx: useEffect -> fetchPosts()
5. Zustand Store: usePostsStore.fetchPosts()
6. services/posts.ts: api.get('/posts')
7. Axios مع التوكن GET يرسل طلب
8. Laravel يستقبل الطلب
9. Sanctum يتحقق من التوكن -> صحيح
10. PostController::index()
11. User->posts()->orderBy('created_at', 'desc')->get()
12. SELECT * FROM posts WHERE user_id = ? ORDER BY created_at DESC
13. PostgreSQL يرجع البيانات
14. PostResource يحول البيانات إلى JSON
15. Laravel يرد: { "data": [{ "id": 1, ... }] }
16. Axios يستقبل الرد
17. Zustand Store يحفظ البيانات
18. React يعيد رسم المكون
19. المستخدم يرى قائمة المنشورات

هذا المسار الكامل يحدث في أقل من ثانية!

الجزء الثالث: قاعدة البيانات (PostgreSQL)

10. ما هي قاعدة البيانات؟

قاعدة البيانات هي نظام لتخزين واسترجاع البيانات بشكل منظم.

بدون قاعدة بيانات، كل البيانات ستضيع عند إغلاق الخادم. قاعدة البيانات:

- تحفظ البيانات بشكل دائم
- تنظم البيانات في جداول
- تسمح بالبحث السريع
- تضمن سلامة البيانات

PostgreSQL يدعم أنواع بيانات متعددة: integer, bigint, varchar(n), text, boolean, date, timestamp, json.

11. لماذا PostgreSQL؟

اخترنا PostgreSQL بدلاً من MySQL للأسباب التالية:

- مجاني تمامًا
- يتبع معايير SQL بدقة
- دعم ممتاز لـ JSON
- أداء ممتاز للعمليات المعقدة
- متوفر مجانًا على Render

التغيير الوحيد الذي طبقناه للتوافق مع PostgreSQL هو تغيير `table->enum$ (...)` إلى `table-$` لأن PostgreSQL Schema Builder لا يدعم `enum`.

12. جدول users (المستخدمين)

يخزن معلومات المستخدمين المسجلين.

العمود	النوع	شرح
id	bigint PK	معرف فريد
name	varchar(255)	اسم المستخدم
email	varchar(255) UNIQUE	البريد الإلكتروني
password	varchar(255)	كلمة المرور (مشفر)
avatar	varchar(255) NULL	صورة المستخدم
tiktok_username	varchar(255) NULL	اسم TikTok
tiktok_open_id	varchar(255) UNIQUE NULL	معرف TikTok
tiktok_access_token	text NULL	توكن TikTok
tiktok_refresh_token	text NULL	توكن التجديد
tiktok_token_expires_at	timestamp NULL	تاريخ انتهاء التوكن
created_at	timestamp	تاريخ الإنشاء
updated_at	timestamp	آخر تحديث

العلاقات: User hasMany Posts, User hasMany DailyStats.

13. جدول posts (المنشورات)

يخزن منشورات TikTok.

العمود	النوع	شرح
id	bigint PK	معرف فريد
user_id	bigint FK -> users.id	صاحب المنشور
description	varchar(255)	وصف المنشور
hashtags	varchar(500) NULL	الهشتاغات
video_url	varchar(255) NULL	رابط الفيديو (Cloudinary)
thumbnail_url	varchar(255) NULL	رابط الصورة المصغرة
views	bigint DEFAULT 0	المشاهدات
likes	bigint DEFAULT 0	الإعجابات
comments	bigint DEFAULT 0	التعليقات
shares	bigint DEFAULT 0	المشاركات
status	varchar(20) DEFAULT 'draft'	الحالة
created_at	timestamp	تاريخ الإنشاء
updated_at	timestamp	آخر تحديث

`user_id` هو مفتاح خارجي (Foreign Key) يرتبط بـ `users.id`. `cascadeOnDelete` يعني إذا حُذف المستخدم، تُحذف منشوراته تلقائيًا.

14. جدول `daily_stats` (الإحصائيات اليومية)

يخزن إحصائيات يومية للرسم البياني.

العمود	النوع	شرح
id	bigint PK	معرف فريد
user_id	bigint FK	معرف المستخدم
date	date	التاريخ
views	bigint DEFAULT 0	المشاهدات
likes	bigint DEFAULT 0	الإعجابات
comments	bigint DEFAULT 0	التعليقات
shares	bigint DEFAULT 0	المشاركات
created_at	timestamp	تاريخ الإنشاء
updated_at	timestamp	آخر تحديث

يوجد `unique(['user_id', 'date'])` لضمان عدم وجود إحصائيتين لنفس المستخدم في نفس اليوم.

15. جدول `personal_access_tokens` (التوكنات)

هذا جدول من Laravel Sanctum لتخزين توكنات المصادقة.

العمود	النوع	شرح
id	bigint PK	معرف فريد
tokenable_type	varchar	نوع المستخدم
tokenable_id	bigint	معرف المستخدم
name	varchar	اسم التوكن
token	varchar(64)	التوكن (مشفر)
abilities	text NULL	الصلاحيات
last_used_at	timestamp NULL	آخر استخدام
expires_at	timestamp NULL	تاريخ الانتهاء
created_at	timestamp	تاريخ الإنشاء
updated_at	timestamp	آخر تحديث

عند تسجيل الدخول، ينشئ Laravel توكنًا جديدًا ويخزنه هنا. عند كل طلب، Sanctum يبحث في هذا الجدول عن التوكن للتحقق من صحته.

16. العلاقات بين الجداول

```

users (1) -----> (N) posts      (مستخدم يملك منشورات متعددة)
users (1) -----> (N) daily_stats (مستخدم يملك إحصائيات يومية متعددة)
users (1) -----> (N) personal_access_tokens (مستخدم يملك توكنات متعددة)
users (1) -----> (N) sessions   (مستخدم يملك جلسات متعددة)

```

17. فهم الـ Migrations

الهجرة (Migration) هي ملف PHP يتحكم في هيكل قاعدة البيانات. بدل إنشاء الجداول يدويًا في PostgreSQL، تكتب الكود مرة واحدة وتشغله.

مميزاتها:

- التحكم بالنسخ (Version Control)
- المشاركة بين المطورين
- التراجع (Rollback)
- النشر التلقائي

الأمر: `php artisan migrate` يشغل كل ال Migrations. الأمر: `php artisan migrate:rollback` يرجع آخر تغيير.

الجزء الرابع: الباك إند (Laravel)

18. ما هو Laravel؟

Laravel هو إطار عمل (Framework) لتطوير تطبيقات الويب بلغة PHP.

مميزاته:

- سهل التعلم، توثيق ممتاز
- Eloquent ORM للتعامل مع قاعدة البيانات
- Artisan CLI (php artisan ...)
- Sanctum للمصادقة
- Migrations لإدارة قاعدة البيانات

19. نمط MVC

Laravel يتبع نمط MVC (Model-View-Controller):

Model -> بيانات (يتواصل مع قاعدة البيانات) View -> عرض (في مشروعنا، React هو ال View) Controller -> منطق (يعالج الطلبات)

في StackFlow، ال View هو React المستقل. Laravel يخدم فقط API JSON.

20. ملف routes/api.php

يحدد كل مسارات API:

```
Route::post('/login', [AuthController::class, 'login']); // عام
Route::prefix('auth/tiktok')->group(function () { // عام
    Route::get('/redirect', [TikTokAuthController::class, 'redirect']);
    Route::get('/callback', [TikTokAuthController::class, 'callback']);
});
Route::middleware('auth:sanctum')->group(function () { // محمي
    Route::post('/logout', ...);
    Route::get('/user', ...);
    Route::get('/dashboard/stats', ...);
    Route::get('/posts', ...);
    Route::post('/posts', ...);
    Route::delete('/posts/{post}', ...);
});
```

21. AuthController

يدير المصادقة:

- `login()`: يتحقق من الإيميل وكلمة المرور، ينشئ توكن Sanctum، يرجع التوكن + بيانات المستخدم
- `logout()`: يحذف التوكن الحالي
- `user()`: يرجع معلومات المستخدم الحالي

22. DashboardController

يجلب إحصائيات لوحة التحكم:

- `stats()`: يحسب مجموع المشاهدات/الإعجابات/التعليقات من جدول posts
- `dailyStats()`: يجلب الإحصائيات اليومية من جدول daily_stats

23. PostController

يدير المنشورات:

- `index()`: يجلب منشورات المستخدم مرتبة تنازليًا
- `store()`: ينشئ منشورًا جديدًا مع رفع الفيديو/الصورة إلى Cloudinary
- `destroy()`: يحذف منشورًا (يتحقق من الملكية أولاً)

24. TikTokAuthController

يدير توثيق TikTok عبر OAuth 2.0:

- `redirect()`: يرجع رابط توثيق TikTok
- `callback()`: يستقبل الـ code من TikTok، يتبادله مع `access_token`، يجلب معلومات المستخدم، ينشئ أو يحدد حساب المستخدم

25. الموديلات (Models)

User

- Traits: HasApiTokens, HasFactory, Notifiable
- `fillable$`: الحقول القابلة للتعبئة
- `hidden: password, tiktok_tokens$` (لا تظهر في JSON)
- casts: password -> hashed, tiktok_token_expires_at -> datetime
- Relations: hasMany(Post), hasMany(DailyStat)

Post

- `fillable: description, hashtags, video_url, thumbnail_url, views, likes, comments, shares, status$`
- casts: views, likes, comments, shares -> integer
- Relations: belongsTo(User)

DailyStat

- `fillable: date, views, likes, comments, shares$`
- Relations: belongsTo(User)

26. Sanctum للمصادقة

Sanctum هو نظام مصادقة خفيف من Laravel لل APIs.

نستخدم Token-based authentication:

- تسجيل دخول - > يحصل على توكن
- يرسل التوكن في Authorization header: Bearer 1|xxx
- Sanctum يتحقق من التوكن في كل طلب

27. TikTokService

مسؤول عن التواصل مع TikTok API:

- getAuthUrl(): بناء رابط توثيق TikTok
- getAccessToken(): تبادل ال code مع access_token
- refreshToken(): تجديد التوكن المنتهي
- getUserInfo(): جلب معلومات المستخدم
- getVideos(): جلب قائمة الفيديو

28. رفع الملفات إلى Cloudinary

قبلًا: التخزين المحلي (يُمسح على Render) بعدًا: التخزين على Cloudinary (سحابة، 25GB مجاني)

```
// قبل
$data['video_url'] = $request->file('video')->store('posts/videos', 'public');

// بعد
$uploadedFile = $request->file('video')->storeOnCloudinary('posts/videos');
$data['video_url'] = $uploadedFile->getSecurePath();
```

29. CORS

CORS يسمح للفرونت إند (Vercel) بالتواصل مع الباك إند (Render).

في config/cors.php:

```
return [
  'paths' => ['api/*', 'sanctum/csrf-cookie'],
  'allowed_methods' => ['*'],
  'allowed_origins' => [env('FRONTEND_URL')],
  'allowed_headers' => ['*'],
  'supports_credentials' => true,
];
```

الجزء الخامس: الفرونت إند (React)

30. ما هو React؟

React هي مكتبة JavaScript لبناء واجهات المستخدم.

مبادئ React:

1. المكونات (Components): تقسيم الواجهة إلى قطع صغيرة
2. JSX: كتابة HTML داخل JavaScript
3. الحالة (State): البيانات التي تتغير
4. Props: بيانات تدخل للمكون من الخارج
5. Hooks: دوال تضيف ميزات (useState, useEffect)

31. main.tsx

نقطة دخول التطبيق. يرسم React داخل العنصر #root في index.html.

```
ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </React.StrictMode>
)
```

App.tsx .32

المكون الرئيسي. يحتوي على التوجيه (Routing) الشرطي:

إذا لم يسجل دخول -> يعرض فقط صفحة Login إذا سجل دخول -> يعرض (Dashboard, Sidebar + Routes (Posts, CreatePost, Settings)

types/index.ts .33

يعرف أنواع البيانات التي يستخدمها التطبيق: User, Post, Stats, DailyStats.

/services .34

طبقة الخدمات للتواصل مع API:

- api.ts: إنشاء Axios instance مع baseURL من VITE_API_URL + interceptor لإضافة التوكن
- auth.ts: login, logout, getUser, getTikTokAuthUrl
- dashboard.ts: getStats, getDailyStats
- posts.ts: getAll, create (FormData), remove

store/index.ts (Zustand) .35

4 مخازن (Stores):

- useAuthStore: user, isAuthenticated, login, logout, fetchUser
- useDashboardStore: stats, dailyStats, fetchStats
- usePostsStore: posts, fetchPosts, addPost, deletePost
- useSettingsStore: language, setLanguage

hooks/useTranslation.ts .36

يقرأ اللغة من useSettingsStore ويرجع ملف JSON المناسب (en.json أو fr.json).

/components .37

مكونات مشتركة:

- Sidebar.tsx: الشريط الجانبي مع روابط التنقل
- StatsCard.tsx: بطاقة إحصائية (title, value, change)

Tailwind CSS .38

إطار عمل CSS يستخدم Utility Classes مباشرة في HTML. أمثلة:

- bg-[#121212]: خلفية داكنة
- text-white: نص أبيض
- p-6: padding 24px
- rounded-xl: border-radius 12px
- flex, items-center, justify-between: تنسيق مرن

الجزء السادس: شرح الصفحات بالتفصيل

39. صفحة Login

تسجيل الدخول بالإيميل/كلمة المرور أو بحساب TikTok.
تستخدم:

- useState لحالة النموذج (email, password, error)
- Zod للتحقق من صحة البيانات
- useAuthStore للدخول
- useNavigate للتوجيه بعد الدخول

40. صفحة Dashboard

لوحة التحكم مع إحصائيات ورسم بياني.
تستخدم:

- useEffect لجلب البيانات عند التحميل
- StatsCard لعرض الإحصائيات

- Recharts (LineChart) للرسم البياني
- useDashboardStore للبيانات

41. صفحة Posts

قائمة المنشورات مع إمكانية الحذف.

تستخدم:

- useEffect لجلب المنشورات
- usePostsStore للبيانات
- Link للانتقال إلى صفحة الإنشاء

42. صفحة CreatePost

إنشاء منشور جديد مع رفع فيديو/صورة.

تستخدم:

- FormData لإرسال الملفات
- useState لحالة النموذج
- (usePostsStore.addPost)
- useNavigate للتوجيه بعد الإنشاء

43. صفحة Settings

الإعدادات الشخصية وتغيير اللغة.

تستخدم:

- useAuthStore للمستخدم الحالي
- useSettingsStore للغة
- localStorage لحفظ اللغة

44. تدفق البيانات الكامل

مثال: مستخدم جديد -> تسجيل دخول -> مشاهدة المنشورات

1. المتصفح يحمل التطبيق
2. App.tsx: isAuthenticated=false -> Login
3. المستخدم يكتب البيانات ويضغط Submit
4. apiAuth.login() -> POST /api/login
5. Laravel يتحقق -> يرجع توكن + بيانات المستخدم
6. localStorage يحفظ التوكن
7. useAuthStore: isAuthenticated=true
8. App.tsx: يعرض التطبيق الكامل
9. المستخدم يضغط Posts
10. useEffect -> fetchPosts() -> GET /api/posts
11. Laravel يجلب المنشورات من PostgreSQL
12. يعرضها في القائمة

الجزء السابع: النشر والتشغيل

45. تحضير الكود للنشر

التغييرات المطبقة:

- gitignore: تجاهل node_modules, dist, .env, vendor
- composer.json: إضافة cloudinary-labs/cloudinary-laravel
- config/cors.php: جديد (السماح للفرونٲ إند)
- config/cloudinary.php: جديد (إعدادات Cloudinary)
- PostController: تغيير التخزين إلى Cloudinary
- PostResource: إزالة Storage::url()
- Migration: تغيير string -> enum لـ PostgreSQL
- Session migration: جديد

46. نشر الفرونت إند على Vercel

1. ادخل New Project -> Vercel.com - اختر GitHub repo
2. Framework: Vite, Root: /, Build: npm install && npm run build, Output: dist
3. Environment Variable: VITE_API_URL=<https://api.onrender.com/api>
4. Deploy

47. نشر الباك إند على Render

1. ادخل New Web Service -> Render.com - اختر GitHub repo
2. Root Directory: backend
3. Build: composer install --no-dev --optimize-autoloader
4. Start: php artisan serve --host=0.0.0.0 --port=\$PORT
5. Plan: Free
6. أضف Environment Variables من env.example
7. Deploy
8. Shell: php artisan migrate

48. PostgreSQL على Render

1. New -> PostgreSQL -> Plan: Free
2. انسخ Internal Database URL
3. أضف القيم ك Environment Variables في Web Service
4. تأكد من DB_CONNECTION=pgsql

49. Cloudinary

1. cloudinary.com -> Sign up free (25GB)
2. انسخ Cloud Name, API Key, API Secret
3. أضفهم ك Environment Variables في Render

TikTok Sandbox .50

TikTok Developer Portal -> Your App -> Sandbox .1

Redirect URIs: <https://api.onrender.com/api/auth/tiktok/callback> .2

Domains: <https://frontend.vercel.app> .3

تم إعداد هذا الكتاب بواسطة AI لتوثيق مشروع StackFlow (TikTok Manager) بالكامل. التاريخ: 13 يوليو 2026
